

High Tech Marketing Text Analysis

Brand: Motorola Razr 2025

Group 12?

Introduction

Explain shit here philippe 8=====D

Sample Selection

We decided to go with multiple sources of text in an attempt to capture multiple perspectives on the Razr and avoid pigeon holing into one corner of the internet which may have skewed results.

Data Sources

We grabbed our shit from Reddit threads about the Motorola Razr 2025 (pain in my ass) and Youtube comments from popular tech youtubers who reviewed the Razr 2025

Sampling representation & Bias

Explain if the sample is representative of all customers, and how platform choice may skew results e.g Reddit chuds are not normal people.

Data Collection

```
fetch_reddit_thread <- function(thread_url, limit = 500, attempts = 5) {  
  api_url <- paste0(thread_url, ".json?limit=", limit)  
  response <- RETRY(  
    "GET",  
    api_url,  
    user_agent(paste0("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) ",  
                      "AppleWebKit/537.36 (KHTML, like Gecko) ",  
                      "Chrome/125.0.0.0 Safari/537.36")),  
    times      = attempts,  
    pause_base = 2,  
    pause_cap  = 20,  
    terminate_on = c(400, 401, 403, 404)  
  )  
}
```

```

if (http_error(response)) {
  stop("HTTP ", status_code(response), " when fetching ", api_url)
}

payload <- fromJSON(
  content(response, as = "text", encoding = "UTF-8"),
  simplifyVector = FALSE
)
post <- payload[[1]]$data$children[[1]]$data

flatten_comments <- function(children) {
  if (!is.list(children)) return(data.frame())
  rows <- list()
  for (child in children) {
    if (!is.null(child$kind) && child$kind == "t1") {
      comment <- child$data
      body_text <- comment$body %||% NA_character_
      if (!is.null(body_text)) body_text <- gsub("[\r\n]+", " ", body_text)
      rows[[length(rows) + 1]] <- data.frame(
        type = "comment",
        id = comment$id %||% NA_character_,
        parent_id = comment$parent_id %||% NA_character_,
        author = comment$author %||% NA_character_,
        body = body_text,
        score = comment$score %||% NA_real_, # keep score for segmentation
        subreddit = comment$subreddit %||% NA_character_, # keep subreddit for segmentation
        stringsAsFactors = FALSE
      )
      replies <- if (is.list(comment$replies) &&
        !is.null(comment$replies$data$children))
        comment$replies$data$children else list()
      if (length(replies) > 0)
        rows <- c(rows, flatten_comments(replies))
    }
  }
  if (length(rows) == 0) return(data.frame())
  rows <- Filter(function(x) is.data.frame(x) && ncol(x) > 0, rows)
  if (length(rows) == 0) return(data.frame())
  tryCatch(
    dplyr::bind_rows(rows),
    error = function(e) {
      cols <- c("type", "id", "parent_id", "author", "body", "score", "subreddit")
      repaired <- lapply(rows, function(r) {
        if (!is.data.frame(r)) return(NULL)
        for (m in setdiff(cols, names(r))) r[[m]] <- NA
        r[cols]
      })
      dplyr::bind_rows(Filter(Negate(is.null), repaired))
    }
  )
}

comments <- flatten_comments(payload[[2]]$data$children %||% list())

```

```

post_body <- post$selftext %||% post$title %||% NA_character_
if (!is.null(post_body)) post_body <- gsub("[\r\n]+", " ", post_body)

post_df <- data.frame(
  type      = "post",
  id        = post$id      %||% NA_character_,
  parent_id = NA_character_,
  author    = post$author %||% NA_character_,
  body      = post_body,
  score     = post$score  %||% NA_real_,
  subreddit = post$subreddit %||% NA_character_,
  stringsAsFactors = FALSE
)

if (!is.data.frame(comments)) comments <- data.frame()
tryCatch(
  dplyr::bind_rows(post_df, comments),
  error = function(e) post_df
)
}

suppressWarnings({
  reddit_cache <- "data/reddit_content.csv"
  cached <- file.exists(reddit_cache)

  if (cached) {
    reddit_all <- tryCatch(
      read.csv(reddit_cache, stringsAsFactors = FALSE),
      error = function(e) { message("Cache read failed: ", conditionMessage(e)); data.frame() }
    )
  }
  else {
    search_config <- list(
      list(sub = "Android",      kw = "motorola razr"),
      list(sub = "Android",      kw = "razr 2025"),
      list(sub = "Android",      kw = "razr flip"),
      list(sub = "Motorola",      kw = "razr"),
      list(sub = "Motorola",      kw = "razr ultra"),
      list(sub = "Motorola",      kw = "razr plus"),
      list(sub = "phones",        kw = "razr"),
      list(sub = "phones",        kw = "motorola"),
      list(sub = "smartphones",   kw = "motorola razr"),
      list(sub = "razr",          kw = "razr"),
      list(sub = "razr",          kw = "motorola"),
      list(sub = "technology",    kw = "motorola razr"),
      list(sub = "technology",    kw = "razr 2025")
    )

    reddit_posts <- dplyr::bind_rows(Filter(
      function(x) is.data.frame(x) && nrow(x) > 0, # add this Filter wrapper
      lapply(search_config, function(cfg) {
        result <- tryCatch({
          Sys.sleep(2)

```

```

    find_thread_urls(keywords = cfg$kw, subreddit = cfg$sub, sort_by = "top")
  }, error = function(e) {
    message("Failed r/", cfg$sub, " / '", cfg$kw, "': ", conditionMessage(e))
    data.frame(url = character()) # always return a data frame, never NULL
  })
  cat("r/", cfg$sub, " / '", cfg$kw, "' ->",
      if (is.data.frame(result)) nrow(result) else "ERROR", "threads\n")
  # Ensure we always return a data frame regardless of what find_thread_urls gives back
  if (!is.data.frame(result)) return(data.frame(url = character()))
  result
})
)) |> dplyr::distinct(url, .keep_all = TRUE)

cat("Total unique threads:", nrow(reddit_posts), "\n")

reddit_max_threads <- 250

if (nrow(reddit_posts) > 0) {
  to_fetch <- head(reddit_posts$url, reddit_max_threads)
  raw_list <- lapply(seq_along(to_fetch), function(i) {
    Sys.sleep(1)
    message("Thread ", i, "/", length(to_fetch))
    tryCatch(
      fetch_reddit_thread(to_fetch[i]),
      error = function(e) {
        message("Failed: ", conditionMessage(e))
        data.frame()
      }
    )
  })
  reddit_all <- dplyr::bind_rows(
    Filter(function(x) is.data.frame(x) && nrow(x) > 0, raw_list)
  )
} else {
  reddit_all <- data.frame()
}

write.csv(reddit_all, reddit_cache, row.names = FALSE)
})
})

cat("Total rows:", nrow(reddit_all), "\n")

```

```
## Total rows: 992
```

```
cat("Posts vs comments:\n")
```

```
## Posts vs comments:
```

```
print(table(reddit_all$type))
```

```
##
```

```
## comment      post
##      765      227
```

```
cat("By subreddit:\n")
```

```
## By subreddit:
```

```
print(table(reddit_all$subreddit))
```

```
##
##      Android      motorola      phones      razr Smartphones
##           6          312          29          608          37
```

```
fetch_youtube_comments <- function(video_id, max_comments = 200, api_key = Sys.getenv("YOUTUBE_KEY")) {
  if (identical(api_key, "")) {
    stop("YOUTUBE_KEY is not set. Load it from .env or your environment before knitting.")
  }

  all_comments <- list()
  page_token <- NULL

  repeat {
    query <- list(
      part      = "snippet",
      videoId   = video_id,
      maxResults = 100,
      textFormat = "plainText",
      key       = api_key
    )

    # Add pagination token if we have one
    if (!is.null(page_token)) {
      query$pageToken <- page_token
    }

    resp <- httr::GET(
      "https://www.googleapis.com/youtube/v3/commentThreads",
      query = query
    )

    if (httr::http_error(resp)) {
      error_body <- tryCatch(
        httr::content(resp, as = "text", encoding = "UTF-8"),
        error = function(e) ""
      )
      message("YouTube API error: ", httr::status_code(resp), if (nzchar(error_body)) paste0(" - ", error_body))
      break
    }

    parsed <- jsonlite::fromJSON(
      httr::content(resp, as = "text", encoding = "UTF-8"),
      flatten = TRUE
    )
  }
}
```

```

)

# Extract top-level comments
items <- parsed$items

if (is.null(items) || nrow(items) == 0) break

batch <- data.frame(
  comment_id = items$id,
  author      = items$snippet.topLevelComment.snippet.authorDisplayName,
  body        = items$snippet.topLevelComment.snippet.textDisplay,
  stringsAsFactors = FALSE
)

all_comments[[length(all_comments) + 1]] <- batch
collected <- sum(sapply(all_comments, nrow))
message("Collected: ", collected, " comments")

# Check for next page
page_token <- parsed$nextPageToken

if (is.null(page_token) || collected >= max_comments) break

Sys.sleep(0.5)
}

if (length(all_comments) == 0) return(data.frame())
dplyr::bind_rows(all_comments) |> dplyr::mutate(source = "youtube");
}

razr_videos <- list(
  mkbhd      = "8om1eJr021U",    # MKBHD Razr 2025 review
  dave2d     = "JTT67FSc8j8",    # ShortCircuit review
  mrmobile   = "YGTkjchlVJk",    # MrMobile review
  phonearena = "b3Ijuf7DHcQ"     # PhoneArena review
)

# to save on api calls, we check if we have a local csv of comments already, if not we fetch and save f
if (file.exists("data/youtube_comments.csv")) {
  youtube_all <- read.csv("data/youtube_comments.csv", stringsAsFactors = FALSE)
  cat("Loaded YouTube comments from CSV:", nrow(youtube_all), "\n")
} else {
  youtube_all <- purrr::map2_dfr(
    razr_videos,
    names(razr_videos),
    ~ fetch_youtube_comments(.x, max_comments = 200) %>%
      dplyr::mutate(channel = .y)
  )
  # Save to CSV for future runs
  write.csv(youtube_all, "data/youtube_comments.csv", row.names = FALSE);
}

```

```
## Loaded YouTube comments from CSV: 704
```

```
cat("YouTube comments collected:", nrow(youtube_all), "\n")
```

```
## YouTube comments collected: 704
```

```
print(table(youtube_all$channel))
```

```
##
##      dave2d      mkbhd      mrmobile phonearena
##      200        200        200          104
```

Exploration

Lets have a gander at the data we collected, and see if theres any thing fun before we clean it up.

```
# reddit
str(reddit_all)
```

```
## 'data.frame':   992 obs. of  7 variables:
## $ type      : chr  "post" "comment" "comment" "comment" ...
## $ id       : chr  "1sz3as2" "oiyrxch" "oj3o8r3" "oj7x93l" ...
## $ parent_id: chr   NA "t3_1sz3as2" "t3_1sz3as2" "t3_1sz3as2" ...
## $ author   : chr  "FragmentedChicken" "BeerorCoffee" "box-art" "z28camaroman" ...
## $ body     : chr   "" "I like the Razr but that pricing is just wild. " "Insane pricing for a phone ...
## $ score    : int   20 10 16 5 4 1 279 78 16 3 ...
## $ subreddit: chr  "Android" "Android" "Android" "Android" ...
```

```
# youtube
str(youtube_all)
```

```
## 'data.frame':   704 obs. of  9 variables:
## $ comment_id: chr  "Ugw-9HIGvJr0Z1iZZyx4AaABAg" "UgwMtrWVbmFWifzG4r54AaABAg" "UgxASY7bEBKG3vLuRsh4...
## $ author    : chr  "@chrishulsey2813" "@brettgirl" "@WalterSobchack-s8n" "@DTow-p1x7x" ...
## $ body      : chr  "Man I know this is old but can you review the newer ultras with 1tb storage and ...
## $ like_count: int   0 0 0 0 0 0 1 0 0 0 ...
## $ reply_count: int  0 0 0 0 0 0 0 0 0 0 ...
## $ date      : chr  "2026-05-03T12:24:10Z" "2026-04-28T15:45:50Z" "2026-04-02T05:27:44Z" "2026-03-30...
## $ source    : chr  "youtube" "youtube" "youtube" "youtube" ...
## $ segment   : chr  "neutral" "neutral" "neutral" "neutral" ...
## $ channel   : chr  "mkbhd" "mkbhd" "mkbhd" "mkbhd" ...
```

Pre-Processing

we have 2 separate data frames for reddit, and youtube, we need to merge them into one big one for the text analysis. We can add a column to indicate the source of each row and then bind them together.

```
##      [,1]
## de      1
## en 1355
```

```
## es      6
## fr      1
## pt      2
## ro      1
## ru      1
```

Side note maybe we dont have to do all of them based on the quality of the data we get, but we should at least explain how we would do them and why they are important.

Filtering & Cleaning

```
#remove non english comments
combined_data <- combined_data |> filter(language == "en")

#remove shit
combined_data$body <- gsub("[^\x01-\x7F]", "", combined_data$body)

#remove empty bodies
combined_data <- combined_data |> filter(body != "")
```

Modeling

Corpus Creation

```
corpus <- quantda::corpus(combined_data, docid_field = "id", text_field = "body")
summary(corpus, 10)
```

Corpus consisting of 1355 documents, showing 10 documents:

```
##
## Text Types Tokens Sentences author score source language
## 1 11 11 1 BeerorCoffee 10 Android en
## 2 14 15 1 box-art 16 Android en
## 3 51 71 3 z28camaroman 5 Android en
## 4 18 22 2 Slowpc 4 Android en
## 5 11 13 1 the-Vibe 1 Android en
## 6 15 19 2 Powerful_District_82 78 motorola en
## 7 17 19 3 LoETR9 16 motorola en
## 8 6 6 1 dscdrivercpm-fr 3 motorola en
## 9 63 84 2 astralsparkle 3 motorola en
## 10 9 9 2 dirtyvu 10 motorola en
```

```
as.character(corpus[ sample(length(corpus), 1) ])
```

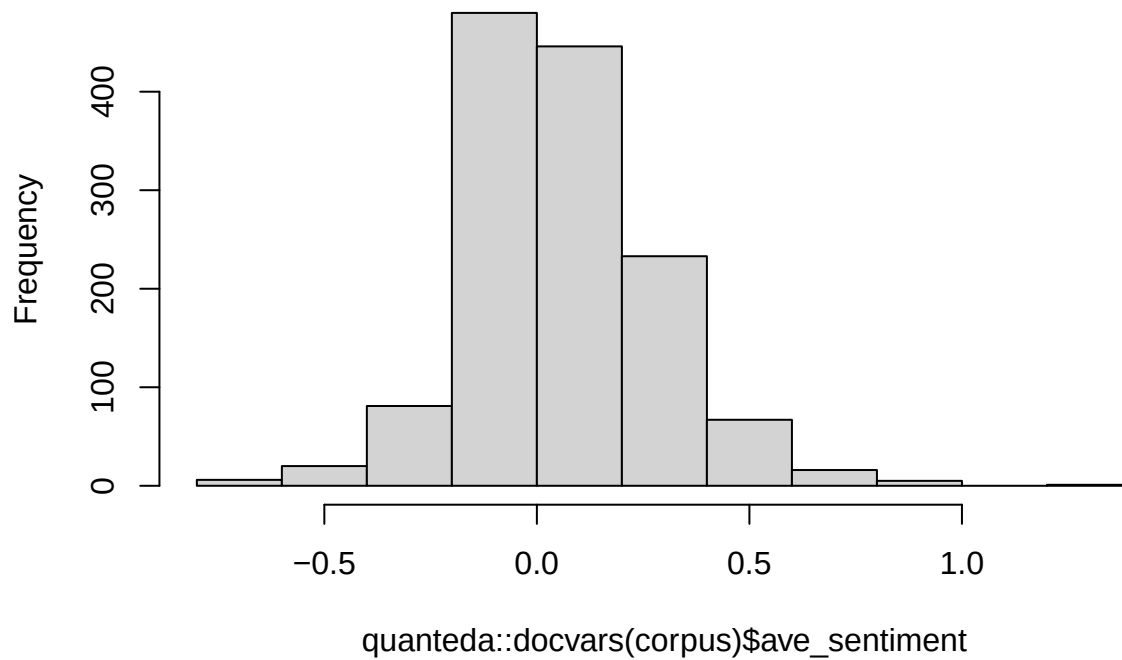
```
##
```

```
## "It shows the pen plus a year of warranty. After trade in, the total price is 747. Motorola is also c
```



```
quanteda::docvars(corpus, field = "ave_sentiment") <- sentimentr::sentiment_by(sentimentr::get_sentences)
hist(quanteda::docvars(corpus)$ave_sentiment)
```

Histogram of quanteda::docvars(corpus)\$ave_sentiment



```
as.character(corpus[ which(quanteda::docvars(corpus)$ave_sentiment == summary(quanteda::docvars(corpus))
```

```
##
## "The crux of the outline is fine but alot of hard force info which turned out very incorrect" 16
```

```
as.character(corpus[ which(quanteda::docvars(corpus)$ave_sentiment == summary(quanteda::docvars(corpus))
```

```
##
## "Just bought this phone it's straight fire, Thought for sure it would able to flip and that was it a
```

Tokenization

Explain how we tokenized the data and why.

```
tokens <- quanteda::tokens(corpus,
  what = "word", # text unit to extract
  remove_punct = TRUE,
  remove_symbols = TRUE,
```

```
remove_numbers = TRUE,  
remove_url = TRUE,  
remove_separators = TRUE,  
split_hyphens = FALSE,  
split_tags = FALSE,  
include_docvars = TRUE,  
padding = FALSE,  
verbose = TRUE)
```

```
## Creating a tokens from a corpus object...  
  
## ...starting tokenization  
  
## ...tokenizing 1 of 1 blocks  
  
## ...preserving hyphens  
  
## ...preserving elisions  
  
## ...preserving social media tags (#, @)  
  
## ...removing separators, punctuation, symbols, numbers, URLs  
  
## ...5,632 unique types  
  
## ...complete, elapsed time: 0.068 seconds.  
  
## Finished constructing tokens from 1,355 documents
```

```
tokens[ sample(length(tokens), 1) ]
```

```
## Tokens consisting of 1 document and 5 docvars.  
## 1192 :  
## [1] "I'm"      "so"      "over"    "Apple"   "Samsung" "and"     "Google"  
## [8] "playing" "it"      "safe"    "when"    "we"  
## [ ... and 16 more ]
```

```
tokens <- quantda::tokens_tolower(tokens)
```

Stop Words Removal

```
quantda::stopwords("en")
```

##	[1]	"i"	"me"	"my"	"myself"	"we"
##	[6]	"our"	"ours"	"ourselves"	"you"	"your"
##	[11]	"yours"	"yourself"	"yourselves"	"he"	"him"
##	[16]	"his"	"himself"	"she"	"her"	"hers"
##	[21]	"herself"	"it"	"its"	"itself"	"they"
##	[26]	"them"	"their"	"theirs"	"themselves"	"what"
##	[31]	"which"	"who"	"whom"	"this"	"that"
##	[36]	"these"	"those"	"am"	"is"	"are"
##	[41]	"was"	"were"	"be"	"been"	"being"
##	[46]	"have"	"has"	"had"	"having"	"do"
##	[51]	"does"	"did"	"doing"	"would"	"should"
##	[56]	"could"	"ought"	"i'm"	"you're"	"he's"
##	[61]	"she's"	"it's"	"we're"	"they're"	"i've"
##	[66]	"you've"	"we've"	"they've"	"i'd"	"you'd"
##	[71]	"he'd"	"she'd"	"we'd"	"they'd"	"i'll"
##	[76]	"you'll"	"he'll"	"she'll"	"we'll"	"they'll"
##	[81]	"isn't"	"aren't"	"wasn't"	"weren't"	"hasn't"
##	[86]	"haven't"	"hadn't"	"doesn't"	"don't"	"didn't"
##	[91]	"won't"	"wouldn't"	"shan't"	"shouldn't"	"can't"
##	[96]	"cannot"	"couldn't"	"mustn't"	"let's"	"that's"
##	[101]	"who's"	"what's"	"here's"	"there's"	"when's"
##	[106]	"where's"	"why's"	"how's"	"a"	"an"
##	[111]	"the"	"and"	"but"	"if"	"or"
##	[116]	"because"	"as"	"until"	"while"	"of"
##	[121]	"at"	"by"	"for"	"with"	"about"
##	[126]	"against"	"between"	"into"	"through"	"during"
##	[131]	"before"	"after"	"above"	"below"	"to"
##	[136]	"from"	"up"	"down"	"in"	"out"
##	[141]	"on"	"off"	"over"	"under"	"again"
##	[146]	"further"	"then"	"once"	"here"	"there"
##	[151]	"when"	"where"	"why"	"how"	"all"
##	[156]	"any"	"both"	"each"	"few"	"more"
##	[161]	"most"	"other"	"some"	"such"	"no"
##	[166]	"nor"	"not"	"only"	"own"	"same"
##	[171]	"so"	"than"	"too"	"very"	"will"

```

custom_stops <- c(
  "razr", "motorola", "phone", "just", "get", "like",
  "one", "will", "can", "also", "even", "still", "want",
  "moto", "device", "smartphone", "im", "dont", "ive", "thats", "got", "amp",
  "u", "ur", "ok", "yeah", "lol", "gonna", "screen", "use", "good", "get", "make", "think", "need", "lo
)
tokens <- quantda::tokens_remove(tokens, pattern = c(quantda::stopwords("en"), custom_stops))

```

Stemming & Lemmatization

```

tokens <- quantda::tokens_replace(tokens, pattern = quantda::featnames(quantda::dfm(tokens)),replacer
)

```

DFM Creation & Frequency Filtering

```
dfm <- quanteda::dfm(tokens)

dfm <- quanteda::dfm_select(dfm, pattern = "^.{3,}$", valuetype = "regex", selection = "keep")

dfm
```

```
## Document-feature matrix of: 1,355 documents, 3,478 features (99.62% sparse) and 5 docvars.
##      features
## docs price wild insane receive upgrade rival samsung ram increase mean
##   1      1      1      0      0      0      0      0      0      0      0
##   2      1      0      1      1      1      1      1      0      0      0
##   3      3      0      0      0      0      0      0      1      1      1
##   4      0      0      0      0      1      0      0      0      0      0
##   5      0      0      0      0      0      0      0      0      0      0
##   6      0      0      0      0      0      0      0      0      0      0
## [ reached max_ndoc ... 1,349 more documents, reached max_nfeat ... 3,468 more features ]
```

```
dfm <- dfm_trim(dfm, min_termfreq = 10)

dfm
```

```
## Document-feature matrix of: 1,355 documents, 463 features (98.14% sparse) and 5 docvars.
##      features
## docs price receive upgrade samsung ram increase mean less add magsafe
##   1      1      0      0      0  0      0  0  0  0  0      0
##   2      1      1      1      1  0      0  0  0  0  0      0
##   3      3      0      0      0  1      1  1  2  1  1      1
##   4      0      0      1      0  0      0  0  0  0  0      0
##   5      0      0      0      0  0  0      0  0  0  0  0      0
##   6      0      0      0      0  0  0      0  0  0  0  0      0
## [ reached max_ndoc ... 1,349 more documents, reached max_nfeat ... 453 more features ]
```

```
topfeatures(dfm)
```

```
##      good      fold      ultra      app      phone      flip      year      camera      use really
##      189      178      152      146      138      134      132      127      124      120
```

```
length_features <- data.frame(count = quanteda::topfeatures(dfm, ncol(dfm))) # create data frame
length_features$length <- nchar(rownames(length_features)) # extract token length
length_features[ which(length_features$length %in% c(1, 2)), ] # filter on token length
```

```
## [1] count length
## <0 rows> (or 0-length row.names)
```

```
dfm <- dfm[ -which((rowSums(dfm)) %in% 0), ]
```

Synonym consolidation

```

# Add this chunk under ## Synonym consolidation
# Simple flat vectors - pattern and replacement must be same length
tokens <- quanteda::tokens_replace(tokens,
  pattern = c(
    # Form factor
    "fold", "foldable", "folding",
    "flip", "flipping",
    # Camera
    "cam", "cams",
    # Battery
    "charging", "charger",
    # Display (already have "screen" as stop word - keep display as synonym)
    "display", "displays",
    # Hinge
    "hinges", "hinging",
    # Price
    "pricing", "priced", "pricey", "expensive", "costly",
    # Competitors
    "galaxy",
    "iphone", "ios", "apple"
  ),
  replacement = c(
    # Form factor
    "fold", "fold", "fold",
    "flip", "flip",
    # Camera
    "camera", "camera",
    # Battery
    "charge", "charge",
    # Display
    "screen", "screen",
    # Hinge
    "hinge", "hinge",
    # Price
    "price", "price", "price", "price", "price",
    # Competitors
    "samsung",
    "apple", "apple", "apple"
  ),
  valuetype = "fixed"
)

```

LDA

```

dtm <- quanteda::convert(dfm, to = "topicmodels")

# Split DTM into train/test for perplexity evaluation
set.seed(42)
n_docs      <- nrow(dtm)
train_idx    <- sample(n_docs, floor(n_docs * 0.8))
dtm_train    <- dtm[train_idx, ]

```

```

dtm_test    <- dtm[-train_idx, ]

# Test k values from 2 to 15
k_values <- 2:15

perplexity_scores <- sapply(k_values, function(k) {
  cat("Fitting k =", k, "\n")
  m <- topicmodels::LDA(
    dtm_train,
    k      = k,
    method = "Gibbs",
    control = list(seed = 42, iter = 1000, burnin = 200)
  )
  topicmodels::perplexity(m, newdata = dtm_test) # evaluate on held-out test set
})

```

```

## Fitting k = 2
## Fitting k = 3
## Fitting k = 4
## Fitting k = 5
## Fitting k = 6
## Fitting k = 7
## Fitting k = 8
## Fitting k = 9
## Fitting k = 10
## Fitting k = 11
## Fitting k = 12
## Fitting k = 13
## Fitting k = 14
## Fitting k = 15

```

```

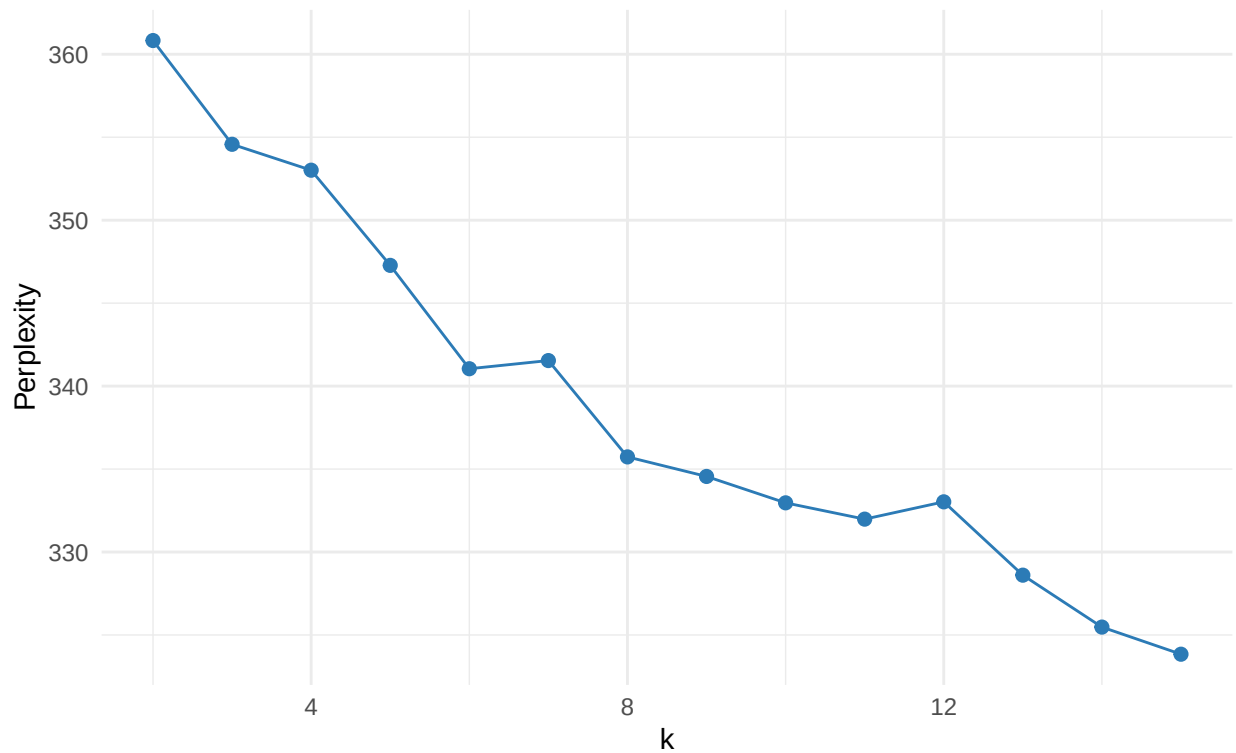
# Plot
perplexity_df <- data.frame(k = k_values, perplexity = perplexity_scores)

ggplot(perplexity_df, aes(x = k, y = perplexity)) +
  geom_line(colour = "#2C7BB6") +
  geom_point(colour = "#2C7BB6", size = 2) +
  labs(title = "Held-out Perplexity by Number of Topics (k)",
       subtitle = "Lower = better generalisation; choose the elbow",
       x = "k", y = "Perplexity") +
  theme_minimal()

```

Held-out Perplexity by Number of Topics (k)

Lower = better generalisation; choose the elbow



```
print(perplexity_df)
```

```
##      k perplexity
## 1    2   360.8274
## 2    3   354.5750
## 3    4   353.0124
## 4    5   347.2775
## 5    6   341.0478
## 6    7   341.5399
## 7    8   335.7333
## 8    9   334.5554
## 9   10   332.9652
## 10  11   331.9826
## 11  12   333.0259
## 12  13   328.6054
## 13  14   325.4768
## 14  15   323.8415
```

```
K <- 7 # choose based on the plot.
```

```
lda_model <- seededlda::textmodel_lda(dfm, k = K)
```

```
# Then analyse sentiment per topic after the fact
topic_assignments <- data.frame(
  doc_id      = docnames(dfm),
```

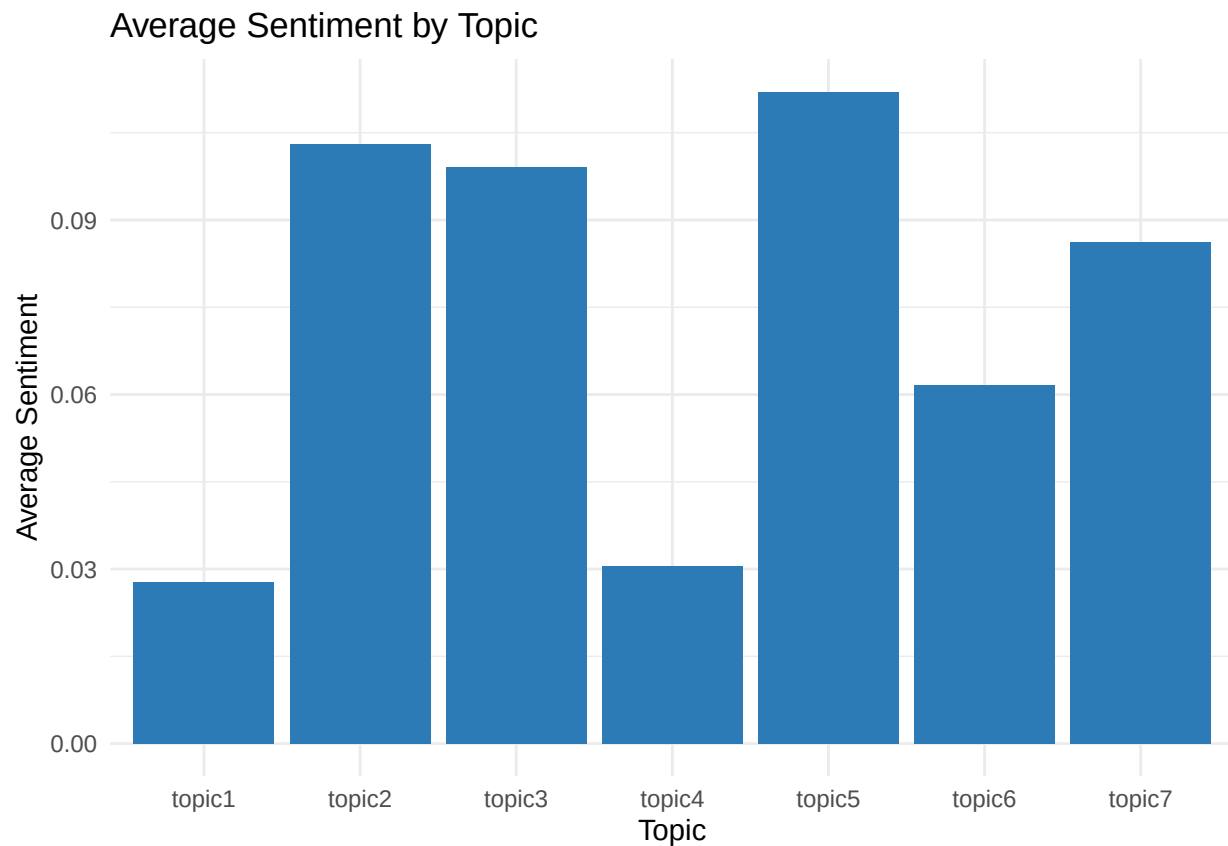
```

topic      = topics(lda_model),
sentiment = docvars(dfm)$ave_sentiment
)

# Average sentiment per topic - this is your comparison
topic_sentiment <- topic_assignments |>
  group_by(topic) |>
  summarise(mean_sentiment = mean(sentiment, na.rm = TRUE),
            n = n())

# Visualize sentiment distribution across topics.
ggplot(topic_sentiment, aes(x = factor(topic), y = mean_sentiment)) +
  geom_bar(stat = "identity", fill = "#2C7BB6") +
  labs(title = "Average Sentiment by Topic",
       x = "Topic", y = "Average Sentiment") +
  theme_minimal()

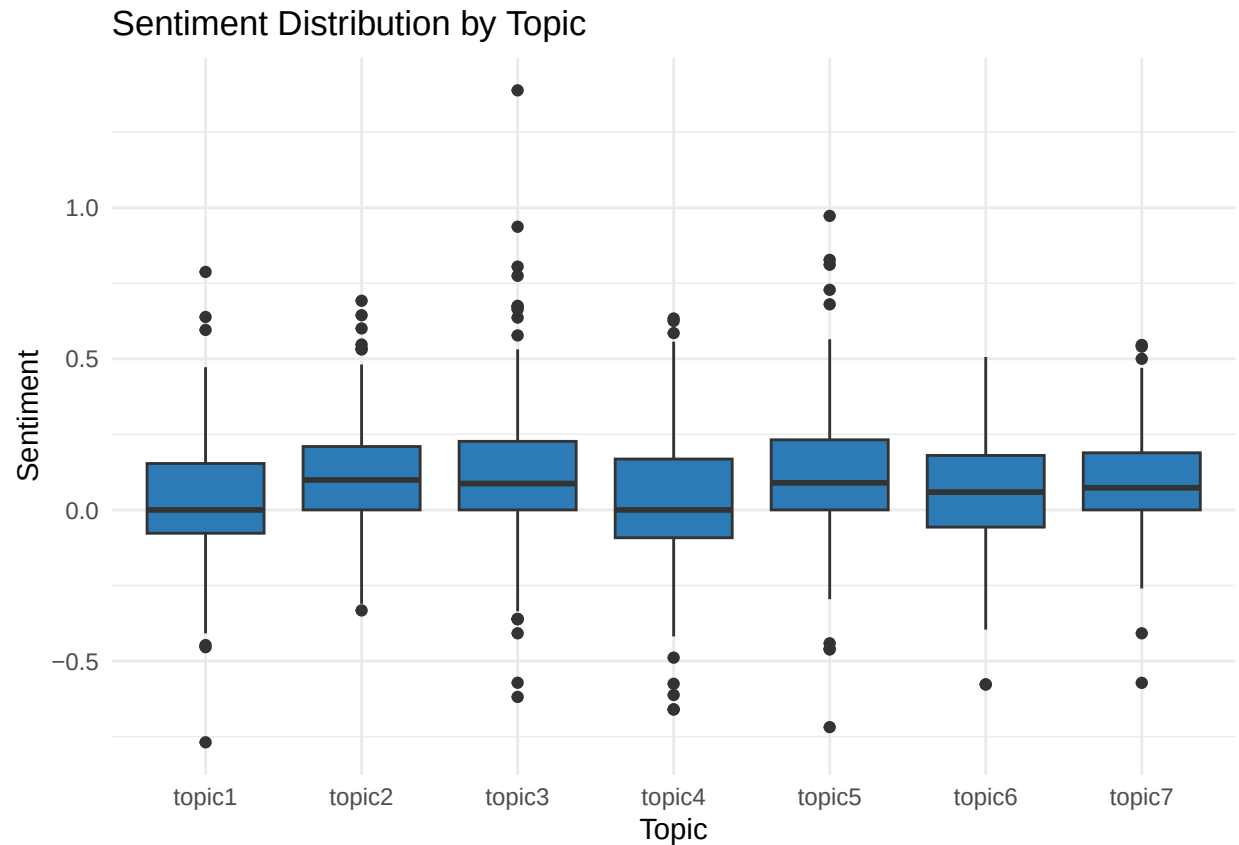
```



```

# You can also look at the distribution of sentiment within each topic:
ggplot(topic_assignments, aes(x = factor(topic), y = sentiment)) +
  geom_boxplot(fill = "#2C7BB6") +
  labs(title = "Sentiment Distribution by Topic",
       x = "Topic", y = "Sentiment") +
  theme_minimal()

```

Improving the model

Stability Analysis

```
seeds <- c(42L, 123L, 999L)

stability_results <- lapply(seeds, function(s) {
  set.seed(s)
  m <- seededlda::textmodel_lda(dfm, k = K)
  terms <- seededlda::terms(m, 5) # matrix: 5 rows (words) x K cols (topics)
  # Collapse each topic's top 5 words into one string per topic
  apply(terms, 2, paste, collapse = ", ") # returns named vector of length K
})

# Each result is a named vector of length K - bind into a clean table
stability_df <- data.frame(
  topic = paste0("Topic ", 1:K),
  seed_42 = stability_results[[1]],
  seed_123 = stability_results[[2]],
  seed_999 = stability_results[[3]],
  row.names = NULL
)

knitr::kable(stability_df,
```

```
col.names = c("Topic", "Seed 42", "Seed 123", "Seed 999"),
caption = "Top 5 words per topic across 3 seeds (stability check)"
```

Table 1: Top 5 words per topic across 3 seeds (stability check)

Topic	Seed 42	Seed 123	Seed 999
Topic 1	fold, mine, wait, day, trade	case, now, get, issue, time	flip, phone, love, back, small
Topic 2	ultra, samsung, year, fold, price	android, year, since, know, update	fold, mine, trade, wait, ship
Topic 3	good, battery, phone, much, iphone	flip, love, small, great, video	case, really, battery, issue, charge
Topic 4	flip, case, year, small, phone	mine, fold, see, wait, day	year, android, iphone, time, update
Topic 5	review, really, great, video, know	good, ultra, battery, camera, model	good, ultra, price, samsung, new
Topic 6	app, work, camera, call, thing	app, work, update, mode, use	app, camera, work, update, mode
Topic 7	update, display, software, support, may	fold, good, price, buy, year	review, video, thing, use, call

Results

```
# Show top 10 words per topic with your human-readable label
topic_terms <- seededlda::terms(lda_model, 10)
knitr::kable(topic_terms, caption = "Top 10 terms per topic")
```

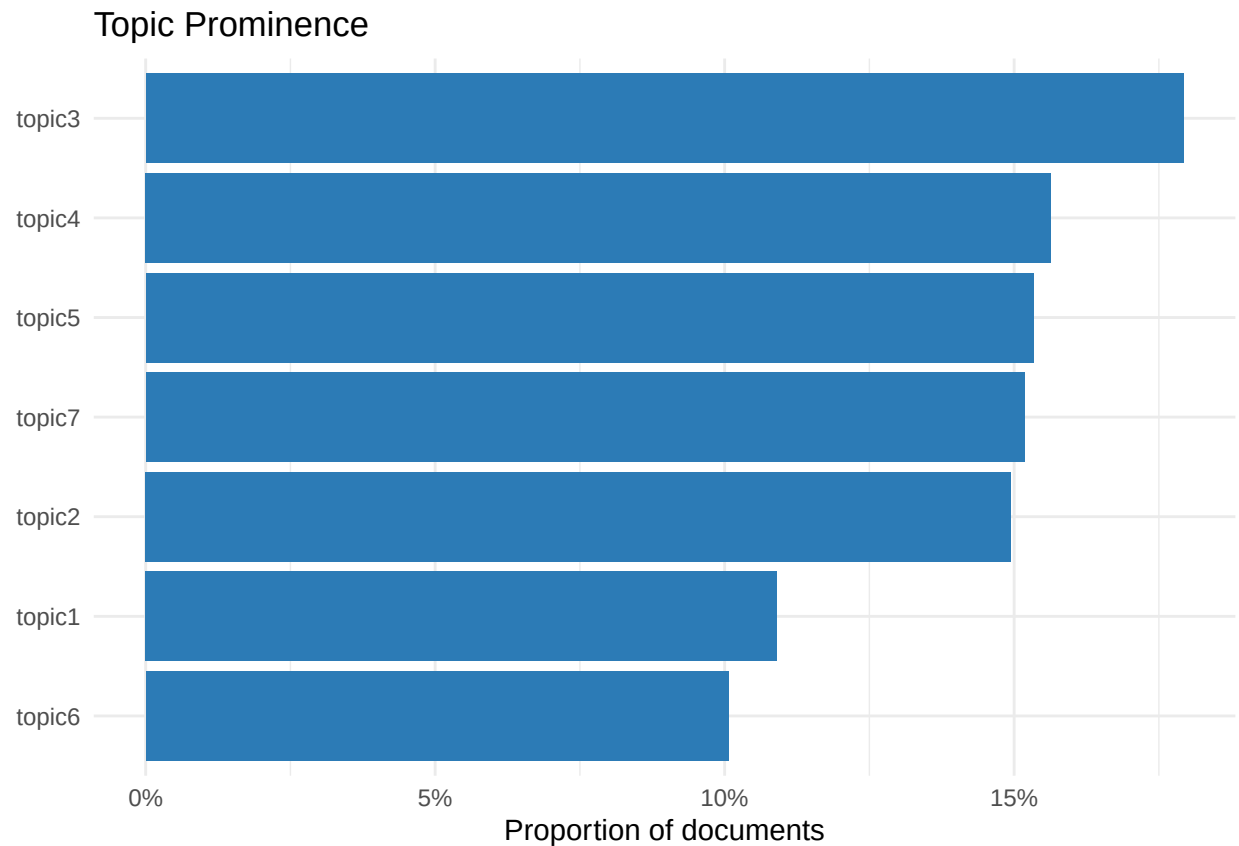
Table 2: Top 10 terms per topic

topic1	topic2	topic3	topic4	topic5	topic6	topic7
app	price	flip	case	good	update	fold
camera	good	phone	time	battery	display	trade
work	ultra	love	android	iphone	support	wait
open	year	great	year	ultra	app	mine
call	plus	small	apple	camera	know	ship
use	buy	review	buy	pixel	keyboard	come
google	new	back	first	phone	hello	see
play	pay	fold	break	charge	help	know
thing	model	video	never	really	mode	day
close	edge	big	find	samsung	external	order

```
# TODO: label each topic based on dominant terms, e.g.:
# topic1 = "Price & Value", topic2 = "Camera Quality", etc.

topic_counts <- as.data.frame(table(topics(lda_model))) |>
  rename(topic = Var1, n = Freq) |>
  mutate(prop = n / sum(n))
```

```
ggplot(topic_counts, aes(x = reorder(topic, prop), y = prop)) +
  geom_col(fill = "#2C7BB6") +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Topic Prominence",
       x = NULL, y = "Proportion of documents") +
  theme_minimal()
```



```
# Merge topic assignments back with source info
topic_source <- topic_assignments |>
  mutate(doc_id = as.integer(doc_id)) |>
  left_join(combined_data |> select(id, source),
            by = c("doc_id" = "id"))

contingency <- table(topic_source$topic, topic_source$source)
knitr::kable(contingency, caption = "Topic distribution by source")
```

Table 3: Topic distribution by source

	Android	dave2d	mkbhd	motorola	mrmobile	phonearena	phones	razr	Smartphones
topic1	0	26	15	18	23	8	0	61	2
topic2	3	19	17	43	27	11	1	60	3
topic3	0	49	75	15	32	20	2	42	2

	Android	dave2d	mkbhd	motorola	mrmobile	phonearena	phones	razr	Smartphones
topic4	1	31	18	15	30	13	4	90	2
topic5	0	24	27	33	29	28	5	50	7
topic6	0	13	11	29	23	6	1	49	1
topic7	1	22	19	38	18	5	2	90	2

```
chi_result <- chisq.test(contingency)
```

```
## Warning in chisq.test(contingency): Chi-squared approximation may be incorrect
```

```
print(chi_result)
```

```
##
## Pearson's Chi-squared test
##
## data:  contingency
## X-squared = 210.61, df = 48, p-value < 2.2e-16
```

```
# Interpret: p < 0.05 means Reddit and YouTube discuss different topics
```

Topic Word Distributions

Sometother shit

Reflection